EKO — Akıllı Sepet Asistanı

Q-Gen Hackathon 2026 — Proje Dokümantasyonu

> **Tek cümleyle:** Alışveriş kararını hem cüzdana hem dünyaya iyi gelecek şekilde dönüştüren, multi-agent mimariye sahip akıllı sepet asistanı.

İçindekiler

1. Proje Vizyonu ve Problem Tanımı
2. Çözüm: Eko Asistanı
3. Sistem Mimarisi
4. Ajan Detayları
5. Pazarlık Mekanizması ve Satıcı-Müşteri Dengesi
6. Wow Faktörleri
7. Teknik Yığın
8. Veri Modelleri
9. API Sözleşmesi
10. Klasör Yapısı
11. Implementasyon Kapsamı (Sepet Sistemi)
12. 6 Günlük Yol Haritası
13. İş Bölümü ve Eş Zamanlı Çalışma Stratejisi
14. Git Stratejisi ve Conflict Önleme
15. README Şablonu
16. 1 Dakikalık Video Senaryosu

---

1. Proje Vizyonu ve Problem Tanımı

Problem

E-ticaret pazarında üç büyük sorun var ve hepsi birbirine bağlı:

Sorun 1 — Cart Abandonment (Sepet Terk Etme) Türkiye'de e-ticaret sepet terk etme oranı ortalama %70 civarındadır. Müşterilerin önemli bir kısmı sepete ürün ekler, ama checkout aşamasında vazgeçer. En büyük üç sebep: fiyat itirazı, beklenmedik kargo ücretleri, ve teslimat süresi memnuniyetsizliği.

Sorun 2 — Hızlı Teslimat Yarışı ve Karbon Maliyeti E-ticaret platformları arasında "ertesi gün teslimat" rekabeti son yıllarda büyük bir lojistik maliyeti ve devasa bir karbon ayak izi yarattı.

Halbuki müşterilerin önemli bir kısmının ürüne **o gün** ihtiyacı yoktur. Ama mevcut sistem onlara bir esneklik sunmuyor.

Sorun 3 — Statik Fiyatlar ve Müzakere Kültürünün Kaybı Türkiye gibi pazarlık kültürü güçlü bir toplumda, e-ticaret sitelerindeki sabit fiyatlar müşteri açısından hayal kırıklığı yaratır. Müşteri "bu fiyat aslında müzakere edilebilir" hissini kaybedince sepetini terk eder veya başka platforma geçer.

Vizyon

Eko, bu üç problemi tek bir multi-agent yapay zeka asistanı ile çözer. Eko, müşterinin yanında bir kişisel alışveriş danışmanı gibi konumlanır; fiyat itirazlarını çözer, çevre dostu teslimat seçenekleriyle hem müşteriye indirim hem satıcıya lojistik tasarrufu sağlar, ve tüm bunları yaparken satıcının kar marjını koruyan akıllı bir politika çerçevesinde çalışır.

Eko'nun değer önerisi:

- **Müşteriye:** Fiyat avantajı, kişiselleştirilmiş öneriler, çevreye katkı bilinci
- **Satıcıya:** Cart abandonment'ta azalma, lojistik maliyetlerinde düşüş, ESG raporlamasına katkı, "Eko Satıcı" prestiji
- **Topluma:** Karbon salınımında azalma, sürdürülebilir e-ticaret kültürüne katkı

---

2. Çözüm: Eko Asistanı

Kullanıcı Yolculuğu

Eko, müşterinin e-ticaret sitesindeki sepet ekranında devreye girer. Müşteri sepete ürünleri ekledikten sonra sağ alt köşede sürekli görünen "Eko ile Konuş" butonuna tıklar. Slide-in olarak sağdan açılan bir chat panelinde Eko ile konuşmaya başlar.

Eko, müşterinin niyetini anlayarak iki farklı moda geçer:

Mod 1 — Pazarlık Modu: Müşteri fiyat itirazı yaparsa Eko, satıcının önceden belirlediği pazarlık politikası çerçevesinde müzakere yapar. İndirim, hediye, bundle veya ücretsiz kargo gibi seçenekler sunar.

Mod 2 — Yeşil Teslimat Modu: Müşteri satın alma kararı verdikten sonra Eko, teslimat tercihi konusunda bir teklif yapar. Konsolide teslimat (birkaç gün bekleme) karşılığında indirim ve karbon tasarrufu sunar.

Müşteri hiçbir aşamada zorlanmaz. İsterse pazarlık etmeden satın alır, isterse standart teslimatı seçer. Eko sadece **fırsat sunar**.

Çok Ajanlı Çalışma Prensibi

Kullanıcı tek bir asistanla (Eko) konuştuğunu sanır, ama arka planda dört ajan koordineli çalışır:

1. **Orkestratör Ajan** — kullanıcı niyetini analiz eder, hangi alt-ajanı çağıracağına karar verir

2. **Pazarlık Ajanı** — fiyat müzakerelerini yürütür
3. **Yeşil Lojistik Ajanı** — teslimat müzakerelerini yürütür
4. **Veri Ajanı** — diğer ajanlara stok, fiyat, marj, karbon hesabı gibi veri sağlar

Bu ajanlar arası iletişim LangGraph'ın state graph yapısı üzerinden gerçekleşir ve A2A (Agent-to-Agent) protokolünün hackathon şartnamesinde geçen kullanımını karşılar.

---

3. Sistem Mimarisi

Yüksek Seviye Mimari

```
┌──────────────────────────────────────────────────────────────────────┐
│                          FRONTEND (React)                            │
│  ┌──────────────────────┐          ┌──────────────────────────┐      │
│  │  Müşteri Sepet UI    │          │  Satıcı Dashboard UI     │      │
│  │  - Ürün listesi      │          │  - Pazarlık ayarları     │      │
│  │  - Eko chat paneli   │          │  - ROI raporu            │      │
│  │  - Karbon görseli    │          │  - Canlı ajan log        │      │
│  │  - Persona seçici    │          │  - Eko Satıcı rozeti     │      │
│  └──────────┬───────────┘          └──────────┬───────────────┘      │
└─────────────┼─────────────────────────────────┼──────────────────────┘
              │                                 │
              │      HTTP / WebSocket           │
              │                                 │
┌─────────────▼─────────────────────────────────▼──────────────────────┐
│                       BACKEND (FastAPI)                              │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                  API Layer (REST + WS)                       │    │
│  │  /chat  /cart  /seller/policy  /seller/stats  /logs          │    │
│  └──────────────────────┬───────────────────────────────────────┘    │
│                         │                                            │
│  ┌──────────────────────▼───────────────────────────────────────┐    │
│  │               LangGraph State Machine                        │    │
│  │  ┌──────────────────────────────────────────────────────┐    │    │
│  │  │            Orkestratör Ajan (Gemini)                 │    │    │
│  │  │   - Intent detection                                 │    │    │
│  │  │   - Routing kararı                                   │    │    │
│  │  │   - State yönetimi                                   │    │    │
│  │  └──────────┬──────────────────────────┬────────────────┘    │    │
│  │             │                          │                     │    │
│  │  ┌──────────▼──────────┐   ┌──────────▼──────────────┐      │    │
│  │  │  Pazarlık Ajanı     │   │  Yeşil Lojistik Ajanı   │      │    │
│  │  │  (Gemini)           │   │  (Gemini)               │      │    │
│  │  └──────────┬──────────┘   └──────────┬──────────────┘      │    │
│  │             │                          │                     │    │
```

```
│  │              └──────────┬──────────┘                          │   │
│  │                         │                                     │   │
│  │   ┌─────────────────────▼──────────────────────┐              │   │
│  │   │          Veri Ajanı (Servis)                 │              │   │
│  │   │   - Stok / Marj / Taban Fiyat                │              │   │
│  │   │   - Bölge Yoğunluğu / Karbon Hesabı          │              │   │
│  │   │   - Müşteri Segmenti / Hafıza                │              │   │
│  │   └─────────────────────┬──────────────────────┘              │   │
│   └────────────────────────┼──────────────────────────────────────┘   │
│                            │                                            │
│   ┌────────────────────────▼──────────────────────────────────────┐   │
│   │            Persistence Layer (SQLite)                          │   │
│   │  products, sellers, customers, sessions, logs                  │   │
│   └────────────────────────────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────────────┘
```

LangGraph State Graph

State graph akışı:

```
[START]
   │
   ▼
[Orkestratör: Intent Detection]
   │
   ├── intent == "price_objection" ──┐
   │                                 ▼
   │                        [Pazarlık Ajanı]
   │                                 │
   │                                 ▼
   │                        [Veri Ajanı: marj/stok/segment]
   │                                 │
   │                                 ▼
   │                        [Müzakere Cevabı]
   │                                 │
   │                                 ▼
   │                        [State Güncelle: negotiation_history]
   │                                 │
   │                                 ▼
   │                                [END]
   │
   ├── intent == "purchase_decision" ─┐
   │                                 ▼
   │                        [Yeşil Lojistik Ajanı]
   │                                 │
   │                                 ▼
   │                        [Veri Ajanı: bölge/karbon]
   │                                 │
   │                                 ▼
   │                        [Yeşil Teklif Cevabı]
   │                                 │
   │                                 ▼
   │                        [State Güncelle: carbon_saved]
   │                                 │
   │                                 ▼
   │                                [END]
   │
   └── intent == "general" ──────────┐
                                     ▼
                        [Genel Cevap (Orkestratör)]
                                     │
                                     ▼
                                    [END]
```

State içeriği:

Field	Tip	Açıklama
`session_id`	string	Oturum kimliği
`customer_id`	string	Müşteri kimliği (persona ID)
`cart_items`	list	Sepetteki ürünler
`cart_total`	float	Sepet toplamı
`negotiation_history`	list	Pazarlık mesajları
`final_discount`	float	Varılan toplam indirim
`final_gifts`	list	Verilen hediyeler
`purchase_confirmed`	bool	Satın alma onayı
`delivery_preference`	string	"express" / "consolidated"
`carbon_saved_kg`	float	Tasarruf edilen CO_2
`customer_segment`	string	"new" / "loyal" / "bargain_hunter"
`negotiation_quota_used`	int	Bu oturumda kullanılan pazarlık hakkı

---

4. Ajan Detayları

4.1. Orkestratör Ajan

Görev: Kullanıcının her mesajını okur, niyetini tespit eder, doğru ajana yönlendirir, gelen cevabı "Eko" kimliğiyle kullanıcıya sunar.

Niyet Kategorileri:

- `greeting` — selamlama, tanışma
- `price_objection` — "pahalı", "indirim", "bütçem yetmiyor", "ucuzlatın"
- `purchase_decision` — "tamam alıyorum", "satın al", "onaylıyorum"
- `delivery_question` — teslimat hakkında soru
- `product_question` — ürün hakkında genel soru
- `general` — diğer

Prompt Yapısı (Sistem Mesajı):

```
Sen "Eko"sun. Bir e-ticaret sitesinde müşterilere yardımcı olan kişisel alışveriş asistanısın.
Görevin: müşterinin mesajını anlayıp doğru aksiyonu almak.
Görevin DEĞİL: doğrudan pazarlık yapmak veya teslimat seçenekleri sunmak.
Bu işleri yapan uzman ajanlara yönlendir.

Kullanıcının mesajını analiz et ve JSON olarak şunu döndür:
{
  "intent": "<kategorilerden biri>",
  "confidence": <0-1 arası>,
  "user_message_summary": "<kısa özet>"
}
```

Önemli: Orkestratör doğrudan kullanıcıya cevap vermez. Routing yapar, cevap diğer ajanlardan gelir.

4.2. Pazarlık Ajanı

Görev: Müşteriyle doğal dilde müzakere yapar. Satıcının izin verdiği sınırlar içinde indirim, hediye, bundle önerisi yapar.

Süreç:

1. Veri Ajanı'ndan sepetteki her ürün için: taban fiyat, kar marjı, stok durumu, satış hızı verisini çeker
2. Müşterinin segment bilgisini alır
3. Müşterinin bu oturumda kaç kez pazarlık yaptığını kontrol eder (kota)
4. Makul bir teklif oluşturur
5. Doğal dilde sunar

Prompt Yapısı:

```
Sen Eko'nun pazarlık modülüsün. Görevin müşteriyle adil bir müzakere yapmak.

Kurallar:
1. Taban fiyatın altına ASLA inme.
2. Bütçe içinde maksimum müşteri memnuniyeti hedefle.
3. Cömert ama satıcı aleyhine olma.
4. Saçma tekliflere kibarca makul aralık öner.
5. En fazla 3 tur müzakere yap.

Müşteri segmenti: {customer_segment}
Pazarlık kotası kalan: {quota_remaining}

Ürün verileri:
{product_data_json}

Müşterinin mesajı: "{user_message}"
Önceki pazarlık geçmişi: {negotiation_history}

Cevap olarak şunu döndür:
{
  "response_text": "<müşteriye gösterilecek doğal dil cevap>",
  "offered_discount_amount": <TL cinsinden>,
  "offered_gifts": [<hediye listesi>],
  "is_final_offer": <true/false>,
  "internal_reasoning": "<karar gerekçesi>"
}
```

4.3. Yeşil Lojistik Ajanı

Görev: Müşteri satın alma kararı verdikten sonra konsolide teslimat teklifini sunar.

Süreç:

1. Veri Ajanı'ndan: müşterinin bölgesindeki diğer siparişlerin tahmini teslimat tarihleri, konsolidasyon fırsatı, karbon hesabını çeker
2. İndirim miktarını hesaplar (kargo tasarrufu × belirli bir oran)
3. Karbon tasarrufunu hesaplar (mesafe × ağırlık × katsayı)
4. Müşteriye ikna edici bir dille sunar (sayı + duygusal hook)

Karbon Hesabı Formülü (MVP):

```
co2_saved_kg = (
    total_weight_kg * delivery_distance_km * 0.0002  # standart teslimat emisyonu
    -
    total_weight_kg * delivery_distance_km * 0.00008  # konsolide emisyonu
)
```

Bu formül basit ama gerçekçi (yaklaşık değer). Production'da Climatiq API kullanılır.

Prompt Yapısı:

```
Sen Eko'nun yeşil lojistik modülüsün. Müşteri satın alma kararı verdi.
Şimdi ona standart teslimat yerine konsolide (geciktirilmiş) teslimat öneriyorsun.

Kurallar:
1. İkna et ama zorlama. Reddedebilir.
2. Hem cüzdana fayda (indirim TL) hem dünyaya fayda (karbon kg) vurgula.
3. Duygusal hook ekle: "X kg = bir ağacın aylık temizlediği hava" gibi.
4. Standart teslimat tarihi de açık tutuluyor — müşteri zorlama hissetmemeli.

Verileri:
- Standart teslimat tarihi: {express_date}
- Konsolide teslimat tarihi: {consolidated_date}
- İndirim TL: {discount_amount}
- Karbon tasarrufu: {co2_saved_kg} kg

Cevap formatı:
{
  "response_text": "<müşteriye gösterilecek doğal dil>",
  "discount_amount": <TL>,
  "co2_saved_kg": <kg>,
  "tree_equivalent": "<X ağaca eşdeğer>",
  "alternative_offered": true
}
```

4.4. Veri Ajanı

Görev: Diğer ajanlara veri sağlayan iç servis. Aslında bir LLM ajanı değil, deterministik bir servistir, ama mimari netliği için "ajan" olarak adlandırılır.

Sunduğu Veriler:

Fonksiyon	Girdi	Çıktı
`get_product_data(product_id)`	ürün ID	taban fiyat, marj, stok, satış hızı
`get_customer_segment(customer_id)`	müşteri ID	"new"/"loyal"/"bargain_hunter"
`get_customer_history(customer_id)`	müşteri ID	son 5 etkileşim özeti
`get_delivery_data(address, weight)`	adres + ağırlık	konsolidasyon fırsatı, tarihler
`calculate_carbon(weight, distance, mode)`	ağırlık + mesafe + mod	kg CO_2
`check_negotiation_quota(customer_id)`	müşteri ID	kalan hak sayısı
`check_seller_budget(seller_id)`	satıcı ID	kalan bütçe

---

5. Pazarlık Mekanizması ve Satıcı-Müşteri Dengesi

Temel Prensip

> **Sistem müşteriye karşı pazarlık etmiyor. Satıcının pazarlama bütçesini akıllıca dağıtıyor.**

Bu prensip jüri sorularına verilecek temel cevabın çekirdeğidir.

Satıcı Tarafı Kontrol Mekanizmaları

Satıcı, dashboard'da kendi pazarlık politikasını belirler:

Ürün Bazlı Ayarlar:

- Pazarlık açık/kapalı toggle
- Maksimum indirim oranı (% veya TL)
- Tercih edilen ödün türleri (indirim/hediye/bundle/kargo)
- Stok hassasiyeti (otomatik daha esnek pazarlık)

Mağaza Bazlı Ayarlar:

- Aylık toplam pazarlık bütçesi
- Hedef minimum kar marjı
- Müşteri segmentine göre strateji (yeni müşteri agresif / sadık nazik)

İzleme ve Kontrol:

- Tüm pazarlık geçmişine okuma erişimi
- Anlık "pazarlık modunu durdur" butonu

- ROI raporu (verilen indirim vs kurtarılan sepet)

Müşteri Tarafı Suistimal Engelleri

- **Pazarlık kotası:** aynı kategoride belirli süre içinde sınırlı pazarlık
- **Tek sepet tek pazarlık:** sepet onaylanana kadar tek müzakere
- **Karşı teklif değerlendirme:** saçma teklife kibar reddedip makul aralık önerme
- **Segment bazlı tavan:** pazarlık avcısı müşterilere mesafeli yaklaşım

Dengeli Algoritma

Her pazarlık anında sistem şunu maksimize eder:

```
fayda = müşteri_kabul_olasılığı × beklenen_kar
```

Burada:

- `müşteri_kabul_olasılığı`: teklif cömertliğiyle artar
- `beklenen_kar`: teklif cömertliğiyle azalır

Sistem bu iki eğrinin kesişim noktasını hedefler.

---

6. Wow Faktörleri

Hackathonda öne çıkacak üç özellik:

Wow 1 — Eko'nun Hafızası

Eko, müşterinin geçmiş etkileşimlerine atıfta bulunur. Mock data ile yapılır:

Örnek:

> *"Geçen sefer de Cuma günkü konsolide teslimatı seçmiştiniz, 1.2 kg CO_2 kurtarmıştınız. Bu hafta da öyle yapmak ister misiniz?"*

Veri Ajanı `get_customer_history` ile son etkileşim özetini çeker. Pazarlık veya Yeşil Lojistik ajanı bu bilgiyi prompt'a ekler.

Wow 2 — Satıcı Tarafı Dashboard

Demo video'da son saniyelerde satıcı dashboard'u gösterilir. İçeriği:

- Bu hafta kapatılan pazarlık sayısı
- Toplam verilen indirim
- Pazarlık olmasa kaybedilecek tahmini sipariş sayısı
- Net ROI hesabı
- "Eko Satıcı" rozeti ve karbon tasarrufu

Bu, B2B değer önerisini somutlaştırır. Pazara açılım puanını uçurur.

Wow 3 — Canlı Ajan Log Paneli

Demo video'sunda en parlak gösterim. Müşteri Eko ile konuşurken, ekranın bir kısmında ajanlar arası mesajlaşma **canlı** akar:

```
[16:42:01] Orkestratör → intent_detection
           result: "price_objection" (confidence: 0.94)
[16:42:01] Orkestratör → Pazarlık Ajanı: ROUTE
[16:42:01] Pazarlık Ajanı → Veri Ajanı: get_product_data
[16:42:01] Veri Ajanı → response: { "base_price": 950, "margin": 0.18, "stock": 47
}
[16:42:02] Pazarlık Ajanı → Veri Ajanı: get_customer_segment
[16:42:02] Veri Ajanı → response: "new"
[16:42:02] Pazarlık Ajanı → karar: indirim 150 TL + hediye kılıf
[16:42:03] Pazarlık Ajanı → Eko: response_ready
```

Bu, A2A protokolünün gerçekten çalıştığının somut kanıtıdır. Jüri bunu görünce "gerçek multi-agent" der.

---

7. Teknik Yığın

Backend

Katman	Teknoloji	Versiyon	Neden
Runtime	Python	3.11+	LangGraph uyumu
API	FastAPI	0.110+	Hızlı, type-safe, ekibin deneyimi var
Agent Framework	LangGraph	0.2+	State-based multi-agent için ideal
LLM	Google Gemini	1.5 Pro / Flash	Hackathon şartı
LLM SDK	google-generativeai	latest	Resmi SDK
DB	SQLite	builtin	MVP için yeter, kurulum yok
ORM	SQLAlchemy	2.0+	Schema yönetimi
Migration	Alembic	latest	Schema değişiklikleri
Validation	Pydantic	2.0+	API ve state validasyonu
WebSocket	FastAPI native	builtin	Canlı log paneli için

Katman	Teknoloji	Versiyon	Neden
Test	pytest	latest	Kritik mantık testleri
Lint	ruff	latest	Hızlı formatleme

Frontend

Katman	Teknoloji	Versiyon	Neden
Framework	React + Vite	18 + 5	Hızlı dev experience
Stil	TailwindCSS	3+	Hızlı UI
Komponent	shadcn/ui	latest	Hazır komponent kütüphanesi
Routing	React Router	6+	Müşteri/Satıcı sayfaları
State	Zustand	latest	Hafif global state
HTTP	Axios	latest	API client
WebSocket	native WebSocket	builtin	Log paneli
Animasyon	Framer Motion	latest	Karbon animasyonu
Icon	Lucide React	latest	Modern icon set
Grafik	Recharts	latest	Satıcı dashboard grafikleri

Geliştirme Araçları

- **Git + GitHub** — versiyon kontrolü
- **Claude Code** — implementasyon hızı için
- **Postman / Thunder Client** — API testi
- **OBS Studio** — video çekimi
- **DaVinci Resolve** (opsiyonel) — video edit

Ortam Değişkenleri (.env)

```bash
```

```
GEMINI_API_KEY=...
DATABASE_URL=sqlite:///./eko.db
SECRET_KEY=...
ENVIRONMENT=development
LOG_LEVEL=INFO
```

---

8. Veri Modelleri

SQLAlchemy Modelleri

Product

```python
class Product(Base):
    __tablename__ = "products"
    id: str  # UUID
    seller_id: str  # FK
    name: str
    category: str
    list_price: float          # listelenen fiyat
    base_price: float          # taban (kırmızı çizgi) fiyat
    margin: float              # kar marjı (0-1)
    stock: int
    sales_velocity: float      # gün başına ortalama satış
    weight_kg: float
    image_url: str
    negotiation_enabled: bool
    max_discount_pct: float    # bu ürün için maksimum indirim
    preferred_concessions: list[str]  # ["discount", "gift", "bundle"]
```

Seller

```python
class Seller(Base):
    __tablename__ = "sellers"
    id: str
    name: str
    monthly_negotiation_budget: float
    monthly_negotiation_spent: float
    min_margin_target: float
    eco_seller_badge: bool
    total_co2_saved_kg: float
```

Customer (Persona)

```python
class Customer(Base):
    __tablename__ = "customers"
    id: str
    name: str
    segment: str  # "new" / "loyal" / "bargain_hunter"
    address: str
    region_code: str
    interaction_history: JSON  # son 5 etkileşim özeti
    total_co2_saved_kg: float
    eco_customer_badge: bool
```

Session

```python
class Session(Base):
    __tablename__ = "sessions"
    id: str
    customer_id: str
    cart_items: JSON
    negotiation_history: JSON
    final_discount: float
    final_gifts: JSON
    purchase_confirmed: bool
    delivery_preference: str
    carbon_saved_kg: float
    negotiation_quota_used: int
    created_at: datetime
    updated_at: datetime
```

AgentLog

```python
class AgentLog(Base):
    __tablename__ = "agent_logs"
    id: int  # auto increment
    session_id: str
    timestamp: datetime
    agent_name: str  # "orchestrator" / "negotiator" / "logistics" / "data"
    action: str
    payload: JSON
```

3 hazır müşteri persona'sı:

1. **Ayşe (new)** — yeni müşteri, fiyat hassasiyetli. Geçmiş etkileşim yok.
2. **Mehmet (loyal)** — 3 yıllık sadık müşteri, çevre duyarlı. Geçmişte konsolide teslimat seçmiş.
3. **Can (bargain_hunter)** — her seferinde pazarlık yapan müşteri. Geçmişte 5 başarılı pazarlık.

8-10 ürün seed data:

- Elektronik (kulaklık, klavye, mouse)
- Giyim (sweatshirt, ayakkabı)
- Ev (yastık, lamba)
- Kitap

1 satıcı: "TechStore"

---

9. API Sözleşmesi

REST Endpoints

Müşteri Tarafı

```
GET    /api/products                          → Ürün listesi
GET    /api/personas                          → Mevcut persona listesi
POST   /api/session/start                     → Yeni oturum başlat
       body: { customer_id }
       returns: { session_id }
GET    /api/session/{session_id}              → Oturum durumu

POST   /api/cart/{session_id}/add             → Sepete ürün ekle
       body: { product_id, quantity }
POST   /api/cart/{session_id}/remove          → Sepetten ürün çıkar
       body: { product_id }
GET    /api/cart/{session_id}                 → Sepet detayı

POST   /api/chat/{session_id}                 → Eko'ya mesaj gönder
       body: { message }
       returns: {
         response_text,
         agent_used,
         offer_details,      # opsiyonel
         carbon_data         # opsiyonel
       }

POST   /api/checkout/{session_id}             → Satın almayı onayla
       returns: { order_id, summary }
```

Satıcı Tarafı

```
GET    /api/seller/{seller_id}/dashboard      → Dashboard verileri
GET    /api/seller/{seller_id}/policy         → Pazarlık politikası
PUT    /api/seller/{seller_id}/policy         → Politikayı güncelle
GET    /api/seller/{seller_id}/stats          → ROI ve istatistikler
GET    /api/seller/{seller_id}/logs           → Geçmiş pazarlık logları
```

Sistem

```
WS     /ws/logs/{session_id}                  → Canlı ajan log streami
GET    /api/health                            → Sağlık kontrolü
```

Örnek İstek-Cevap

POST /api/chat/{session_id}

İstek:

```
json
```

```
{
  "message": "Bu sepet bana 1800 TL'ye olur mu?"
}
```

Cevap:

```json
{
  "response_text": "Anlıyorum, sepetiniz şu an 2150 TL. Sizin için en iyi teklifim ş
  "agent_used": "negotiator",
  "offer_details": {
    "original_price": 2150,
    "offered_price": 1900,
    "discount_amount": 250,
    "gifts": ["Kulaklık kılıfı"],
    "free_shipping": true,
    "is_final": false
  },
  "carbon_data": null,
  "agent_logs": [...]
}
```

---

10. Klasör Yapısı

```
eko/
├── README.md
├── docs/
│   ├── ARCHITECTURE.md          # Mimari diyagram
│   ├── API.md                   # API dokümantasyonu
│   ├── DEMO_GUIDE.md            # Demo akışı
│   └── ROADMAP.md               # v2 vizyonu
│
├── backend/
│   ├── pyproject.toml           # Python bağımlılıkları
│   ├── .env.example
│   ├── alembic.ini
│   ├── alembic/
│   │   └── versions/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py              # FastAPI uygulaması
│   │   ├── config.py            # Ortam değişkenleri
```

```
│   │   ├── database.py           # SQLAlchemy session
│   │   │
│   │   ├── models/               # SQLAlchemy modelleri
│   │   │   ├── __init__.py
│   │   │   ├── product.py
│   │   │   ├── seller.py
│   │   │   ├── customer.py
│   │   │   ├── session.py
│   │   │   └── agent_log.py
│   │   │
│   │   ├── schemas/              # Pydantic schemas
│   │   │   ├── __init__.py
│   │   │   ├── product.py
│   │   │   ├── chat.py
│   │   │   ├── seller.py
│   │   │   └── session.py
│   │   │
│   │   ├── api/                  # REST endpoints
│   │   │   ├── __init__.py
│   │   │   ├── products.py
│   │   │   ├── cart.py
│   │   │   ├── chat.py
│   │   │   ├── checkout.py
│   │   │   ├── seller.py
│   │   │   └── ws_logs.py
│   │   │
│   │   ├── agents/               # AGENT KATMANI
│   │   │   ├── __init__.py
│   │   │   ├── graph.py          # LangGraph state graph
│   │   │   ├── state.py          # AgentState tanımı
│   │   │   ├── orchestrator.py   # Orkestratör Ajan
│   │   │   ├── negotiator.py     # Pazarlık Ajanı
│   │   │   ├── logistics.py      # Yeşil Lojistik Ajanı
│   │   │   ├── data_agent.py     # Veri Ajanı (servis)
│   │   │   ├── prompts.py        # Tüm prompt'lar
│   │   │   └── llm.py            # Gemini wrapper
│   │   │
│   │   ├── services/             # İş mantığı servisleri
│   │   │   ├── __init__.py
│   │   │   ├── carbon.py         # Karbon hesabı
│   │   │   ├── pricing.py        # Fiyat/marj hesabı
│   │   │   ├── segmentation.py   # Müşteri segmenti
│   │   │   └── logging.py        # Agent log yazımı
│   │   │
│   │   └── seed/                 # Seed data
│   │       ├── seed_data.py
│   │       ├── products.json
```

```
│   │       ├── customers.json
│   │       └── sellers.json
│   │
│   └── tests/
│       ├── test_carbon.py
│       ├── test_pricing.py
│       └── test_agents.py
│
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   ├── index.html
│   ├── public/
│   │   └── assets/
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── api/
│       │   ├── client.ts          # Axios instance
│       │   ├── products.ts
│       │   ├── chat.ts
│       │   ├── cart.ts
│       │   └── seller.ts
│       │
│       ├── store/                 # Zustand stores
│       │   ├── cartStore.ts
│       │   ├── sessionStore.ts
│       │   └── chatStore.ts
│       │
│       ├── components/
│       │   ├── ui/                # shadcn komponentleri
│       │   ├── customer/
│       │   │   ├── ProductCard.tsx
│       │   │   ├── CartPanel.tsx
│       │   │   ├── EkoChat.tsx
│       │   │   ├── OfferCard.tsx
│       │   │   ├── CarbonAnimation.tsx
│       │   │   └── PersonaSelector.tsx
│       │   ├── seller/
│       │   │   ├── DashboardLayout.tsx
│       │   │   ├── PolicyEditor.tsx
│       │   │   ├── ROIReport.tsx
│       │   │   └── EkoBadge.tsx
│       │   └── shared/
│       │       ├── AgentLogPanel.tsx   # Wow 3
```

```
│       │       └── Header.tsx
│       │
│       ├── pages/
│       │   ├── CustomerPage.tsx
│       │   ├── SellerDashboardPage.tsx
│       │   └── HomePage.tsx
│       │
│       ├── hooks/
│       │   ├── useChat.ts
│       │   ├── useWebSocket.ts
│       │   └── useCart.ts
│       │
│       └── utils/
│           ├── format.ts
│           └── constants.ts
│
├── .github/
│   └── workflows/
│       └── ci.yml                # Opsiyonel CI
├── .gitignore
└── docker-compose.yml            # Opsiyonel
```

---

11. Implementasyon Kapsamı (Sepet Sistemi)

Her özellik üç sepetten birine girer:

Sepet 1 — Tam Implementasyon (~%70 zaman)

Bunlar gerçek mantıkla çalışır:

1. Müşteri sepet ekranı (gerçek ürünler, gerçek toplam)
2. Eko chat arayüzü (gerçekten Gemini'ye bağlı)
3. Orkestratör Ajan (gerçek intent detection)
4. Pazarlık Ajanı (gerçek müzakere, taban fiyat kontrolüyle)
5. Yeşil Lojistik Ajanı (gerçek teklif üretimi)
6. Veri Ajanı (mock DB'den gerçek veri çeker)
7. Taban fiyat ve marj kontrolü (gerçek hesap)
8. Karbon hesabı (basit ama gerçek formül)
9. Karbon görselleştirmesi (gerçek sayıya göre)
10. LangGraph state yönetimi (gerçek state graph)
11. **Wow 3** — Canlı ajan log paneli (WebSocket ile gerçek)

Mock veriyle çalışan gerçek mantık:

1. Müşteri segmentasyonu (persona seçiminden gelir, ama backend gerçekten kullanır)
2. **Wow 1** — Eko hafızası (mock geçmiş, gerçek okuma ve atıf)
3. Pazarlık kotası (gerçek kontrol, başlangıç değeri mock)
4. Stok ve satış hızı verileri (mock data, gerçek karar girdi)

Sepet 3 — Sadece Görsel / Anlatım (~%10 zaman)

Backend'de çalışmıyor, sadece UI'da görünüyor veya README'de anlatılıyor:

1. Satıcı dashboard finansal raporları (sabit rakamlar)
2. Aylık pazarlık bütçesi grafiği (statik progress bar)
3. **Wow 2** — Eko Satıcı rozeti (görsel, gerçek hesap yok)
4. Sadakat puanı detayları (mock değer)
5. Çoklu satıcı yönetimi (README roadmap)
6. Climatiq API entegrasyonu (README roadmap)

---

12. 6 Günlük Yol Haritası

Gün 1 — Temeller ve Paralel Setup

Sabah (Ortak):

- Repo oluşturma, .gitignore, README iskeleti
- Branch stratejisinin belirlenmesi
- Bu dokümanı okuma ve uyum sağlama
- Klasör yapısının iskeletini commit'leme

Öğleden Sonra (Paralel):

- Geliştirici A: Backend setup (FastAPI, SQLAlchemy, Alembic, seed data)
- Geliştirici B: Frontend setup (Vite, Tailwind, shadcn, ana layout)

Gün Sonu Hedefi: Her iki taraf da hello-world seviyesinde çalışır halde. Backend health endpoint döner, frontend ana sayfayı gösterir.

Gün 2 — Çekirdek Yapı

Geliştirici A (Backend):

- Veri modelleri ve seed data tamamlanır
- Veri Ajanı (deterministik servis) yazılır

- Gemini wrapper (`llm.py`)
- Orkestratör Ajanı'nın iskeleti

Geliştirici B (Frontend):

- Ürün listesi sayfası
- Sepet komponenti
- Persona seçici komponenti
- Eko chat arayüzünün iskeleti (henüz API'ye bağlı değil)

Gün Sonu Hedefi: Backend'de Veri Ajanı çalışır, Gemini'ye basit prompt atılabilir. Frontend'de ürünler listelenir, sepete eklenir.

Gün 3 — Ajanlar ve Bağlantı

Geliştirici A (Backend):

- Pazarlık Ajanı tamamen yazılır
- Yeşil Lojistik Ajanı tamamen yazılır
- LangGraph state graph kurulur
- Chat endpoint (`/api/chat`) yazılır
- Agent logging mekanizması

Geliştirici B (Frontend):

- Eko chat ile backend chat endpoint bağlantısı
- Mesaj gönderme/alma akışı
- Sepete ürün ekleme/çıkarma API bağlantısı
- Karbon animasyonu komponenti

Gün Sonu Hedefi: İlk uçtan uca akış çalışır. Müşteri chat'ten mesaj atar, Pazarlık Ajanı cevap verir.

Gün 4 — Wow Faktörleri ve Satıcı Tarafı

Geliştirici A (Backend):

- WebSocket endpoint (`/ws/logs/{session_id}`)
- Wow 1: Eko hafızası implementasyonu
- Satıcı endpoint'leri (`/api/seller/...`)
- Stats hesaplama mantığı

Geliştirici B (Frontend):

- Wow 3: Canlı ajan log paneli (WebSocket bağlantısı)
- Satıcı dashboard sayfası

- ROI Report komponenti
- Wow 2: Eko Satıcı rozeti görseli

Gün Sonu Hedefi: Tüm Wow faktörleri görünür durumda. Satıcı dashboard'u erişilebilir.

Gün 5 — Cila ve Test

Ortak Görevler:

- Uçtan uca tüm akışları test etme
- UI cilası (animasyonlar, geçişler, mikro etkileşimler)
- Karbon animasyonunun parlatılması
- Persona bazlı senaryoların hazırlanması
- README'nin başlangıç versiyonu
- Mimari diyagramın görselleştirilmesi (Excalidraw / draw.io)

Gün Sonu Hedefi: Demo'ya hazır, bug'sız çalışan bir ürün.

Gün 6 — Video, Dokümantasyon, Son Rötuş

Sabah:

- README'nin nihai hali
- Mimari diyagramının README'ye yerleştirilmesi
- Demo GIF veya screenshot'ların hazırlanması

Öğleden Sonra:

- 1 dakikalık video senaryosunun yazılması
- Ekran kaydı (OBS) ve video çekimi
- Video edit (kesim, altyazı, müzik)
- Final commit ve push

Akşam:

- Son testler
- Teslim

---

13. İş Bölümü ve Eş Zamanlı Çalışma Stratejisi

Temel Prensip: Sorumluluk Alanları Kesişmemeli

Eş zamanlı çalışırken conflict'in en büyük sebebi aynı dosyayı iki kişinin değiştirmesidir. Bu dokümanda **dosya bazlı sorumluluk** netleştirilmiştir.

Geliştirici A — "Ajan & Backend Mimarı"

Sorumlu Olduğu Alanlar:

📁 `backend/app/agents/*` — Tüm ajan kodu (yalnız bu kişi dokunur) 📁 `backend/app/services/*` — İş mantığı servisleri 📁 `backend/app/models/*` — Veritabanı modelleri 📁 `backend/alembic/*` — Migrationlar 📁 `backend/app/seed/*` — Seed data

Ortak Yazma Alanları (dikkatli):

📁 `backend/app/api/*` — REST endpoint'ler (paylaşımlı) 📁 `backend/app/schemas/*` — Pydantic schemas (paylaşımlı ama her dosya tek kişi)

Görev Listesi:

Gün	Görev	Çıktı
1	FastAPI + SQLAlchemy setup, Alembic init	Çalışan health endpoint
1	Seed data hazırlama (8 ürün, 3 persona, 1 satıcı)	`seed_data.py`
2	Veri Ajanı (deterministik servis)	`data_agent.py`
2	Gemini wrapper	`llm.py`
2	Orkestratör Ajan iskelet	`orchestrator.py`
3	Pazarlık Ajanı tam implementasyon	`negotiator.py`
3	Yeşil Lojistik Ajanı tam implementasyon	`logistics.py`
3	LangGraph state graph	`graph.py`
3	Chat endpoint	`api/chat.py`
4	WebSocket log endpoint	`api/ws_logs.py`
4	Eko hafıza mekanizması	`services/segmentation.py`
4	Satıcı endpoint'leri	`api/seller.py`
5	Test ve hata düzeltme	-
6	README backend kısımları	`docs/ARCHITECTURE.md`

Geliştirici B — "Frontend & UX Lideri"

Sorumlu Olduğu Alanlar:

📁 `frontend/src/*` — Tüm frontend kodu (yalnız bu kişi dokunur) 📁 `frontend/src/api/*` — API client (Geliştirici A'nın sözleşmesine göre) 📁 `frontend/src/components/customer/*` — Müşteri komponentleri 📁 `frontend/src/components/seller/*` — Satıcı komponentleri 📁 `frontend/src/pages/*` — Sayfa komponentleri

Görev Listesi:

Gün	Görev	Çıktı
1	Vite + Tailwind + shadcn setup	Çalışan ana sayfa
1	Ana layout (Header, Router)	`App.tsx`
2	Ürün listesi sayfası	`ProductCard.tsx`, `CustomerPage.tsx`
2	Sepet paneli	`CartPanel.tsx`
2	Persona seçici	`PersonaSelector.tsx`
2	Eko chat arayüzü (UI only)	`EkoChat.tsx`
3	Eko chat API bağlantısı	`api/chat.ts`
3	Karbon animasyon komponenti	`CarbonAnimation.tsx`
3	Offer card komponenti	`OfferCard.tsx`
4	Satıcı dashboard layout	`DashboardLayout.tsx`
4	ROI Report komponenti	`ROIReport.tsx`
4	Eko Satıcı rozeti	`EkoBadge.tsx`
4	Canlı ajan log paneli	`AgentLogPanel.tsx`
5	UI cilası, animasyonlar	-
6	Demo video çekimi	`demo.mp4`

Ortak Görevler (Pair Programming)

Bu görevler birlikte oturularak yapılır, conflict riski yok çünkü tek seans:

- Gün 1 sabah: Repo init, README iskelet, klasör yapısı
- Gün 5: End-to-end test (her ikisi de açık olur, sırayla test eder)
- Gün 6: Video senaryosu yazma, video çekimi, final README

Günlük Stand-up (15 dakika):

- Her gün 09:30'da kısa görüşme
- Dün ne yaptın, bugün ne yapacaksın, takıldığın yer var mı

API Sözleşmesi Kuralı:

- Geliştirici A bir endpoint yazmadan önce schema'sını Geliştirici B ile teyit eder
- Geliştirici B endpoint'i kullanırken sözleşmeye uyar
- Sözleşme değişiklikleri Slack/Discord/WhatsApp'tan duyurulur

Mock API Stratejisi:

- Geliştirici A endpoint'i tamamlamadan önce Geliştirici B mock cevaplarla başlar
- Bu, Geliştirici B'yi engellemez
- Backend hazır olunca tek satır URL değişikliği ile geçilir

---

14. Git Stratejisi ve Conflict Önleme

Branch Stratejisi

```
main                    → her zaman çalışır halde olmalı (deploy ready)
  ├── develop           → günlük entegrasyon dalı
        ├── backend/agents
        ├── backend/api
        ├── backend/seed
        ├── frontend/customer
        ├── frontend/seller
        └── frontend/wow-features
```

Kural 1: Doğrudan `main` veya `develop`'a commit yok. Her şey feature branch'ten gelir.

Kural 2: Geliştirici A `backend/*` prefix'li branch'ler kullanır. Geliştirici B `frontend/*` prefix'li branch'ler kullanır. Bu sayede branch isim çakışması olmaz.

Kural 3: Her gün sonunda kendi branch'inden `develop`'a PR açılır. Sabah ilk iş `develop`'tan rebase ile güncel veri alınır.

Commit Disiplini

Commit mesaj formatı:

```
[scope] Kısa açıklama

Opsiyonel detay
```

Örnek:

```
[agents] Pazarlık Ajanı temel müzakere mantığı
[frontend] Persona seçici komponenti
[api] Chat endpoint state validasyonu eklendi
[fix] Karbon hesabı negative olabilme bug'ı düzeltildi
```

Conflict Önleme Kuralları

Kural A — Dosya Sahipliği:

Her dosyanın bir **sahibi** vardır (yukarıdaki iş bölümü tablosuna bakın). Sahibi olmayan dosyaya dokunmadan önce sahibine sorulur.

İstisnalar (paylaşımlı dosyalar):

- `README.md` — her ikisi de yazabilir, ama farklı bölümler
- `docs/*` — her ikisi de yazabilir, ama farklı dosyalar
- `backend/app/api/*` — her endpoint farklı dosyada, dosya bazlı sahiplik

Kural B — Schema Önce Sözleşme:

API endpoint'leri değişmeden önce iki kişi de görür. `schemas/*` dosyaları kritik — değişiklik öncesi haber verilir.

Kural C — Migration Tek Elden:

Tüm Alembic migration'ları Geliştirici A yapar. Geliştirici B veri modeline dokunmaz. Eğer Geliştirici B'nin bir field eklemesi gerekiyorsa, Geliştirici A'ya iletir.

Kural D — package.json ve pyproject.toml:

Bağımlılık ekleme her zaman commit edilir ve karşı tarafa bildirilir. Karşı taraf `git pull` sonrası `pip install` veya `npm install` çalıştırır.

Günlük Akış

Sabah:

1. `git checkout develop && git pull`
2. `git checkout -b <branch-adı>` (yeni feature için)
3. Çalış

Akşam:

1. `git add . && git commit -m "..."`
2. `git push origin <branch-adı>`
3. GitHub'da PR aç (target: develop)
4. Karşı tarafa "review için hazır" haber ver
5. Onaydan sonra merge

Sabah Tekrar:

1. `git checkout develop && git pull` (gece merge edilenler gelir)
2. Eski feature branch'i sil
3. Yeni branch'le devam

Acil Durum Protokolleri

"Conflict çıktı, ne yapacağım?"

- Panik yapma
- `git status` ile çakışan dosyaları gör
- Eğer dosya senin sahipliğindeyse, kendi versiyonunu koru (`git checkout --ours`)
- Eğer karşı tarafınsa, onun versiyonunu al (`git checkout --theirs`)
- Eğer paylaşımlıysa, karşı tarafla konuş, manuel birleştir
- `git add . && git commit`

"Pull yapamıyorum, local'de uncommitted değişiklik var"

- `git stash`
- `git pull`
- `git stash pop`
- Yine conflict çıkarsa, manuel çöz

---

15. README Şablonu

`README.md` aşağıdaki yapıda olur. Bu Geliştirici A ve B'nin Gün 6'da birlikte yazacağı şeydir:

```markdown
```

Eko — Akıllı Sepet Asistanı

> Cüzdana iyi, dünyaya iyi: multi-agent yapay zeka ile e-ticarette yeni nesil alışve

[Demo Video](link) | [Mimari](docs/ARCHITECTURE.md) | [Roadmap](docs/ROADMAP.md)

Hızlı Bakış

[Demo GIF buraya gelir]

Eko, e-ticaret sepet ekranında devreye giren bir multi-agent yapay zeka asistanıdır.
Müşteriyle fiyat müzakeresi yapar, çevre dostu teslimat tercihlerine yönlendirir.

Problem ve Çözüm

[Kısa, çarpıcı]

Mimari

[Mimari diyagramı görseli]

Multi-Agent Yapısı

- **Orkestratör Ajan:** ...
- **Pazarlık Ajanı:** ...
- **Yeşil Lojistik Ajanı:** ...
- **Veri Ajanı:** ...

Teknik Yığın

- Backend: FastAPI, LangGraph, Gemini 1.5
- Frontend: React, Vite, Tailwind, shadcn/ui
- DB: SQLite

Kurulum

\`\`\`bash
Backend
cd backend
pip install -e .
alembic upgrade head
python -m app.seed.seed_data
uvicorn app.main:app --reload

Frontend
cd frontend

```
npm install
npm run dev
\`\`\`

## MVP Scope ve Roadmap

### MVP'de Çalışan
- ...

### MVP'de Simüle Edilen
- ...

### v2 Roadmap
- Climatiq gerçek karbon API
- Trendyol/Hepsiburada eklentisi
- Mobil uygulama

## Ekip

- **[İsim]** — Backend & Agents
- **[İsim]** — Frontend & UX

## Lisans

MIT
```

---

16. 1 Dakikalık Video Senaryosu

Senaryo İskelet (60 saniye)

0:00-0:05 — Hook (Problem) *Ekran: hızlı kesik klipler — sepet terk eden müşteri, dolu kamyonlar, "%70 cart abandonment" yazısı* **Voice-over:** *"E-ticaret pazarında her 10 sepetin 7'si terk ediliyor. Sebebi mi? Fiyat, hız, teslimat."*

0:05-0:10 — Çözüm Tanıtımı *Ekran: Eko logosu, ana sayfa açılıyor* **Voice-over:** *"Tanışın: Eko. Cüzdana iyi, dünyaya iyi yapay zeka alışveriş asistanı."*

0:10-0:30 — Pazarlık Akışı (Canlı Demo) *Ekran: müşteri sepete giriyor, Eko ile konuşuyor Görüntü: "1800 TL'ye olur mu?" → Eko cevaplıyor: "1900 + ücretsiz kargo + hediye" Yan panelde* ***Wow 3*** *canlı ajan logları akıyor* **Voice-over:** *"Eko, satıcının kar marjını koruyan akıllı bir politika içinde müşteriyle pazarlık yapıyor."*

0:30-0:45 — Yeşil Teslimat Akışı *Ekran: müşteri "tamam alıyorum" diyor Eko: "3 gün beklerseniz 47 TL indirim + 2.3 kg CO_2 tasarruf" Karbon animasyonu canlı çıkıyor: "Bu yıl kurtardığınız ağaç*

sayısı: 3" **Voice-over:** *"Sonra çevre dostu teslimat. Müşteri kazanır, satıcı lojistik tasarrufu yapar, dünya bir nefes alır."*

0:45-0:55 — Satıcı Tarafı (Wow 2) *Ekran: Satıcı dashboard'u açılıyor ROI rakamları, "Eko Satıcı" rozeti, karbon tasarrufu metriği* **Voice-over:** *"Satıcı tarafında: gerçek ROI, ölçülebilir karbon tasarrufu, Eko Satıcı prestiji."*

0:55-1:00 — Kapanış *Ekran: Eko logosu + slogan Yazı: "Multi-agent. Gemini powered. Made for Q-Gen 2026."* **Voice-over:** *"Eko. Akıllı sepetin yeni adı."*

Video Teknik Notlar

- **Format:** 1080p, MP4
- **Ses:** Net voice-over (telefon mikrofonu olabilir ama sessiz odada), arka planda hafif lo-fi müzik
- **Kesim:** Hızlı, dinamik. Her 3-5 saniyede yeni kesim.
- **Altyazı:** Türkçe altyazı şart. Voice-over olmasa bile videoyu anlayabilmeli.
- **Çekim aracı:** OBS Studio (ücretsiz)
- **Edit aracı:** DaVinci Resolve (ücretsiz) veya CapCut (basit)
- **Çekim sayısı:** En az 3-5 deneme çek, en iyisini seç

Çekim Öncesi Hazırlık

- Bilgisayar bildirim sesleri kapatılır
- Tarayıcıda gereksiz tab'lar kapatılır
- Demo data hazır (her senaryo için)
- Voice-over önce yazılır, sonra okunur
- Sahneler arası kararlı geçişler için OBS scene'leri önceden ayarlanır

---

Son Notlar

Bu dokümantasyon ekibin uyum içinde çalışması için yeterli düzeyde detaylıdır. Çalışma sırasında karşılaşacağınız küçük sorular için hızlı senkronizasyon (mesajlaşma veya 5 dakikalık görüşme) yeterli olur.

Hatırlatma: Hackathon kazanan ekipler kodun hacmiyle değil, gösterilen düşüncenin olgunluğuyla kazanır. Az şey yapın, mükemmel gösterin. Wow faktörlerini ihmal etmeyin. Video'nun ilk 10 saniyesi her şeyden önemli.

İyi şanslar.